Linear Attention Variants and Their Equivalences to Recurrent State Updates

The central claim of this concept is that many "linear attention" mechanisms are recurrent memory systems expressed in a form that can also be trained in parallel. Their main innovation is not simply replacing the Transformer's quadratic attention with a faster formula. They change how a model stores context: instead of retaining every previous key–value pair, the sequence is compressed into a fixed-size state that is updated as new tokens arrive.

In causal SoftMax attention, the output at position $t$compares the current query with all previous keys:

$$y_t = \sum_{i \le t} \text{softmax}\,(q_t^\top k_i) v_i.$$

This gives flexible content-based retrieval, but requires computation and memory that grow with sequence length. Linear attention introduces a feature map ϕso that the similarity function factorizes:

$$\kappa(q, k) = \phi(q)^\top \phi(k).$$

Associativity then allows the sequence dimension to be removed from the inner computation. Define a matrix-valued memory and a normalization vector:

$$S_t = S_{t-1} + v_t \phi(k_t)^\top,\ z_t = z_{t-1} + \phi(k_t).$$

The output becomes:

$$y_t = \frac{S_t \phi(q_t)}{z_t^\top \phi(q_t)}.$$

This is exactly a recurrent state update: the model reads the current state, writes a new key–value association, and queries the updated state. The original Linear Transformer formulation showed that this structure enables linear-time autoregressive inference and constant-size recurrent memory, while parallel prefix-scan methods retain efficient training across a whole sequence Katharopoulos et al., 2020.

The equivalence is broader than one implementation. Schlag, Irie and Schmidhuber formally connected linearized self-attention to fast-weight programmers: networks that dynamically write outer products into temporary weights and retrieve information through matrix multiplication Schlag et al., 2021. In this view, keys act as addresses, values act as content, and the accumulated matrix is an associative memory. Delta-rule variants improve on purely additive writing by first retrieving what the state currently associates with a key, then correcting it toward the new value. Gated variants add input-dependent forgetting, allowing the system to preserve, overwrite or decay information selectively.

Architecture	Context mechanism	Training/inference profile	Main trade-off
SoftMax Transformer	Explicit pairwise token interactions	Highly parallel training; growing KV cache at inference	Strong retrieval, quadratic attention and memory
Linear Transformer	Feature-mapped outer-product state	Parallelizable training; recurrent linear-time inference	Fixed-state compression and possible interference
RetNet / GLA	Decayed or gated recurrent state	Parallel, recurrent and chunkwise forms	Better forgetting and long-context behaviour, but more complex design

RetNet makes the recurrence explicit through a retention mechanism with decay. It provides parallel, recurrent and chunkwise computation paths, aiming to preserve Transformer-like training while offering constant-memory inference Sun et al., 2023. Gated Linear Attention (GLA) adds data-dependent gates to the recurrent state and reports competitive language-modelling results against Transformer and other linear-time baselines. Its authors also report strong length generalization beyond the 2K-token training length. However, they emphasize an important systems caveat: an asymptotically cheaper algorithm can still be slower than optimized SoftMax attention if its memory movement and GPU kernels are poorly designed Yang et al., 2024.

BDH, or Dragon Hatchling, is a useful substrate for understanding this design space. It is a post-Transformer sequence architecture based on locally interacting neuron-like units, sparse positive activations, low-rank communication and recurrent associative state. Its GPU formulation includes linear attention in a large feature or neuron space Kosowski et al., 2025. BDH therefore gives the recurrence a structural interpretation: state is not merely a cache optimization, but a distributed memory whose connections are continually adjusted.

BDH-CQ extends the BDH family for in-context reasoning. Demonstrations update a recurrent contextual memory, after which the query is processed through repeated latent-space computation rather than a generated chain of natural-language reasoning tokens. The BDH-CQ paper reports 29.5% pass@2 on the public ARC-AGI-1 evaluation at a computed cost of $0.00070 per task, and describes a black-box reproduction by affiliated auditors Engdahl et al., 2026. This is evidence for a particular BDH-CQ system, not proof that every linear-attention model reasons efficiently or matches a SoftMax Transformer.

The most important limitation is that algebraic equivalence does not imply behavioural equivalence. A recurrent state has finite capacity. When many keys are compressed into the same matrix, unrelated associations can interfere; additive updates may overwrite neither obsolete information nor contradictory evidence. Gating, decay and delta rules address this problem but introduce their own optimization and hardware costs. Linear attention should therefore be understood as a family of recurrent state-update mechanisms with a favourable computational shape—not as a universally lossless replacement for SoftMax attention. The next decisive question is whether larger or more structured states can preserve exact retrieval and compositional reasoning over long, adversarial sequences while remaining faster in real deployments.